



10.1 安全高阶特性

全密态数据库

- 软硬融合全密态概述
- 使用gsql操作软硬融合全密态数据库
- 使用JDBC驱动操作软硬融合全密态数据库

软硬融合全密态数据库

- 全密态数据库意在解决数据全生命周期的隐私保护问题，使得系统无论在何种业务场景和环境下，数据在传输、运算以及存储的各个环节始终都处于密文状态。当数据拥有者在客户端完成数据加密并发送给服务端后，在攻击者借助系统脆弱点窃取用户数据的状态下仍然无法获得有效的价值信息，从而起到保护数据隐私的能力。
- 全密态数据库目前支持两种连接方式：gsqI连接和jdbc连接。

软硬融合全密态 - 特性描述 (1)

- 从用户视角来看，整个密态等值查询的主要功能分为三部分，主要通过新增的KeyTool工具以及openGauss的增强gsqclient客户端来实现。
- 首先是客户端密钥管理功能，用户借助新增的KeyTool工具来实现CMK的生成、销毁和更新，同时支持密钥导入导出。通过该KeyTool工具的导入导出功能，CMK可以在不同的客户端间进行传输；同时，KeyTool实现了单个客户端侧的密钥管理，通过配置管理文件，可以对密钥的存储、更新进行管理。
- 其次该特性提供密钥创建功能和加密表创建功能，通过新增SQL语法“CREATE CLINET MASTER KEY”和“CREATE COLUMN ENCRYPTION KEY”实现在数据库侧记录和管理CMK和CEK元信息，CMK和CEK信息被记录在新增的系统表中。通过扩展“CREATE TABLE”语法为表的每一个敏感信息列指定列加密密钥和加密算法，方便后续数据密文存储。

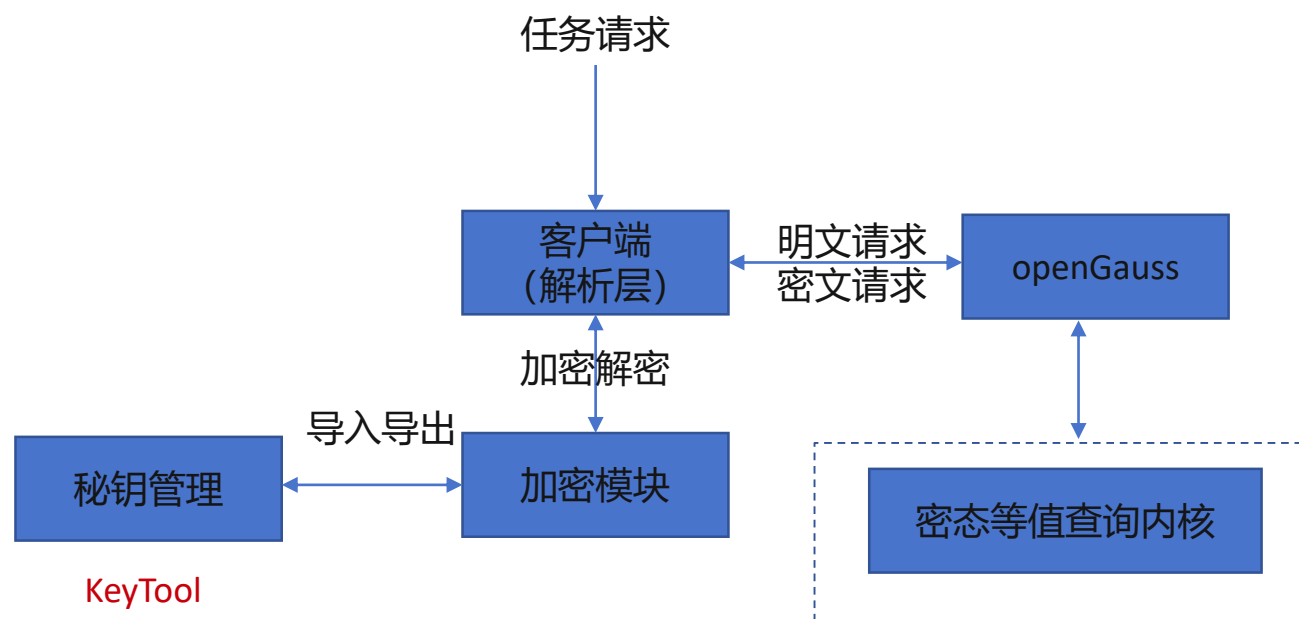
软硬融合全密态 - 特性描述 (2)

- 最后该特性对用户提供的密态等值查询能力，该部分功能是整个特性的核心，虽然用户对整个密文查询无感知，但是在指定数据的加密信息后，针对该敏感数据的查询将受限于当前密态等值查询的支持规格。
- 从整体视角来看，该特性所实现的功能主要是为了用户基于敏感数据保护的诉求来存储管理数据并基于密文数据信息实现查询任务。

软硬融合全密态 - 特性约束

- 不支持不同数据类型之间的隐式转换。
- 不支持不同数据类型密文间的集合操作。
- 不支持加密列创建范围分区。
- 不支持从加密表到文件之间的copy操作。
- 不支持包括排序、范围查询以及模糊查询等在内的除等值以外的各种密态查询。
- 不支持多条SQL语句一起执行，insert into语句多批次执行场景不受此条约束限制。
- 密态数据库对长度为零的空字符串不进行加密。

全密态等值查询流程



- 三层密钥管理机制：根密钥（RK）、主密钥（CMK）和列加密密钥（CEK）。
- 客户端完成数据的加密和解密，服务器完成密态数据计算。
- 不需要加密的字段仍然是明文处理。

使用gsql操作软硬融合全密态数据库 (1)

- 以操作系统用户omm登录CN所在主机。
 - 执行以下命令打开密态开关，连接密态数据库。

```
gsql -p PORT postgres -r -C
```

- 创建客户端主密钥CMK和列加密密钥CEK。
 - 创建客户端加密主密钥(CMK)

```
openGauss=# CREATE CLIENT MASTER KEY ImgCMK1 WITH (KEY_STORE = localkms, KEY_PATH = "key_path_value1", ALGORITHM = RSA_2048);
openGauss=# CREATE CLIENT MASTER KEY ImgCMK WITH (KEY_STORE = localkms, KEY_PATH = "key_path_value2", ALGORITHM = RSA_2048);
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK1 WITH VALUES (CLIENT_MASTER_KEY = ImgCMK1, ALGORITHM =
AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK WITH VALUES (CLIENT_MASTER_KEY = ImgCMK, ALGORITHM =
AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
```

使用gsqI操作软硬融合全密态数据库 (2)

- 查询存储密钥信息的系统表结果如下。

```
openGauss=# SELECT * FROM gs_client_global_keys;
global_key_name | key_namespace | key_owner | key_acl |      create_date
-----+-----+-----+-----+-----
imgcmk1        |      2200 |      10 |      | 2021-04-21 11:04:00.656617
imgcmk         |      2200 |      10 |      | 2021-04-21 11:04:05.389746
(2 rows)
```

```
openGauss=# SELECT column_key_name,column_key_distributed_id ,global_key_id,key_owner FROM gs_column_keys;
column_key_name | column_key_distributed_id | global_key_id | key_owner
-----+-----+-----+-----
imgcek1        |      760411027 |      16392 |      10
imgcek         |      3618369306 |      16398 |      10
(2 rows)
```

使用gsql操作软硬融合全密态数据库 (3)

- 创建加密表

```
openGauss=# CREATE TABLE creditcard_info (id_number int, name text encrypted with (column_encryption_key = ImgCEK, encryption_type = DETERMINISTIC), credit_card varchar(19) encrypted with (column_encryption_key = ImgCEK1, encryption_type = DETERMINISTIC));
NOTICE: The 'DISTRIBUTE BY' clause is not specified. Using 'id_number' as the distribution column by default.
HINT: Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
```

- 插入数据

```
openGauss=# INSERT INTO creditcard_info VALUES (1,'joe','6217986500001288393');
INSERT 0 1
openGauss=# INSERT INTO creditcard_info VALUES (2, 'joy','6219985678349800033');
INSERT 0 1
openGauss=# select * from creditcard_info where name = 'joe';
 id_number | name | credit_card
-----+-----+-----
1 | joe | 6217986500001288393
(1 row)
```

使用gsql操作软硬融合全密态数据库 (4)

```
openGauss=# select id_number,name from creditcard_info;
id_number | name
-----+-----
1 |
\x011aefabd754ded0a536a96664790622487c4d366d313aec5839e410a46d29cba96a60e4831000000ee79056a114c9a6c041bb552b78052e912a8b730609142074c63791abebd0d38
2 |
\x011aefabd76853108eb406c0f90e7c773b71648fa6e2b8028cf634b49aec65b4fcfb376f3531000000f7471c8686682de215d09aa87113f6fb03884be2031ef4dd967afc6f7901646b
(2 rows)
```

- 对加密表进行alter和update操作。

```
openGauss=# ALTER TABLE creditcard_info ADD COLUMN age int ENCRYPTED WITH (COLUMN_ENCRYPTION_KEY = ImgCEK,
ENCPTION_TYPE = DETERMINISTIC); ALTER TABLE openGauss=# \d creditcard_info(2 rows)
```

使用JDBC驱动操作软硬融合全密态数据库

- JDBC支持密态数据库相关操作，需要设置enable_ce=1
- 使用JDBC操作密态数据库时，一个数据库连接对象对应一个线程，否则，不同线程变更可能导致冲突。
- 使用JDBC操作密态数据库时，不同connection对密态配置数据有变更，由客户端调用isValid方法保证connection能够持有变更后的密态配置数据，此时需要保证参数refreshClientEncryption为1(默认值为1)，在单客户端操作密态数据场景下，refreshClientEncryption参数可以设置为0。